

Diseño y análisis de Algoritmos

Problemas

Prof. Gilberto Gutiérrez R.

Primavera 2025

Problema 1 Resuelva las siguientes ecuaciones de recurrencias y obtenga O y Ω de cada ecuación.

1. $T(n) = T(n-1) + 2n, T(0) = 1$
2. $T(n) = 3T(n-1) + 2, T(0) = 0$
3. $f(n) = f(n-1) + f(n-2), f(0) = 0, f(1) = 1$
4. $3t(n+1) = 4t(n), n \geq 1, t(1) = 5$
5. $f(n) = 2f(n-1) + 2n, f(0) = 1$

Problema 2: Asuma el siguiente algoritmo para ordenar un arreglo de enteros de tamaño n -

```
newSort(A,i,n) {  
  if (i<=n) {  
    k = MINIMO(A,i, n) // obtiene la posición k del menor valor de A[i..n]  
    intercambia(A,i,k)  
    newSort(A,i+1,n)  
  }  
}
```

Asuma como operación relevante la comparación entre números enteros.

1. Plantee la Ec. de recurrencia de newSort
2. Obtenga la complejidad del algoritmo newSort
3. ¿ Es mejor newSort que SortBasico (visto en clases) ? ¿ Por qué?

Problema 3: Obtener MAX. Este problema es muy sencillo y consiste en obtener el valor máximo de una secuencia de enteros almacenados en un arreglo no ordenado. Asuma el algoritmo (ALG1) básico para calcular el máximo. Es decir, se toma el primer elemento y se asume el máximo. Luego se recorre el arreglo (una pasada) y se va modificando convenientemente el máximo con los valores del arreglo. Considere ahora el siguiente algoritmo (ALG2)

```

int MAX(A, n) {
if(el tamaño de A == 1) return A[1]
else {
    sea A1 = A[1...n/2]
    sea A2 = A[n/2+1...n]
    m1 = MAX(A1, n/2)
    m2 = MAX(A2, n/2)
    if(m1 > m2) return m1
    else return m2;
}
}

```

Asuma como operación relevante la comparación entre valores numéricos y que n es potencia de 2, es decir, $n = 2^k$, para algún $k = 0, 1, \dots$

1. Obtenga la complejidad del algoritmo ALG1
2. Plantee y resuelva la Ec. de recurrencia de ALG2
3. ¿Cuál de los dos algoritmos es mejor ?

Problema 3: Búsqueda del k -ésimo de un conjunto. Asuma un conjunto de números enteros (no hay repetidos), el k -ésimo menor elemento del conjunto corresponde al elemento que ocupa la posición k al ordenar el conjunto en forma ascendente (de menor a mayor). Una forma obvia de solucionar este problema es ordenar los elementos del conjunto y retornar el elemento que está en la posición k . Llamamos A1 a este algoritmo. Sin embargo, esta solución es de complejidad temporal (peor caso) $O(n \log_2 n)$ y que corresponde al costo de ordenar los elementos. Existe otro algoritmo (dividir para reinar), que llamamos A2, para encontrar el k -ésimo menor que no necesita ordenar el arreglo. El algoritmo es el siguiente:

```

int k-esimo(S, k) {
if(k > |S|) return Error
else {
    separar S en tres conjuntos:
    S1 con los elementos de S < S[k]
    S2 con los elementos de S = S[k]
    S3 con los elementos de S > S[k]
    if(|S1| > k) return k-esimo(S1,k)
    if(|S1 U S2| >= k) return S[k]
    else return k-esimo(S3,k-|S1 U S2|)
}
}

```

Notar que el algoritmo A2 particiona el conjunto de manera muy parecida a como lo hace el algoritmo QuickSort.

1. Plantee la ecuación de recurrencia de A2 para determinar la cantidad de comparaciones (operación relevante) que realiza.
2. Es mejor A2 que A1. ¿Por qué ?

Problema 4 Suponga que sobre el río Ñuble hay n embarcaderos. En cada uno de ellos se puede arrendar un bote que permite ir a cualquier otro embarcadero río abajo (es imposible ir río arriba). Existe una tabla T de tarifas que indica el costo del viaje del embarcadero i al j para cualquier embarcadero de partida i y cualquier embarcadero de llegada j más abajo en el río ($i < j$). Puede suceder que un viaje de i a j sea más caro que una sucesión de viajes más cortos, en cuyo caso se tomaría un primer bote hasta un embarcadero k y un segundo bote para continuar a partir de k . No hay costo adicional por cambiar de bote. Asumiendo una tabla de tarifas T como

	1	2	3	4	5
1	-	3	5	6	8
2	-	-	4	7	10
3	-	-	-	4	5
4	-	-	-	-	5
5	-	-	-	-	-

Por ejemplo, para ir directamente de 2 a 5 cuesta 10, pero resulta más barato ir de 2 a 3 (costo 4) y luego de 3 a 5 (costo 5) con un costo total del viaje igual a 9.

Una forma directa de calcular recursivamente el costo mínimo para ir de un embarcadero i a un embarcadero j es mediante la siguiente fórmula:

$$C(i, j) = \begin{cases} 0 & \text{si } i = j \\ \min_{i < k \leq j} \{T(i, k) + C(k, j)\} & \text{si } i < j \end{cases}$$

1. Escriba un algoritmo recursivo para calcular el costo $C(i, j)$.
2. Determine la complejidad de su algoritmo recursivo.
3. Diseñe un algoritmo de programación dinámica que determine el costo mínimo para cada par de embarcaderos i, j ($i < j$). **Indicación:** Asuma una matriz $C(i, j)$ para almacenar el costo mínimo para ir del embarcadero i al j . Notar que todas las celdas de la diagonal principal ($i = j$) son cero. Luego considere ir llenando la matriz (usando la fórmula dada para $C(i, j)$) diagonal por diagonal desde la principal hacia la celda $[1, n]$ (esquina superior derecha de la matriz).
4. Para el ejemplo de la matriz T , dada arriba, aplique su algoritmo de programación dinámica para obtener la matriz C .

Problema 5 Suponga que dos equipos A y B se enfrentan para ver quien es el primero en ganar $n > 0$ partidos de béisbol y por lo tanto quien gana el encuentro. Suponga que A y B son igualmente competente, de modo que cada uno tiene un 50% de posibilidades de ganar cualquier partido.

Sea $p(i, j)$ la probabilidad de que A gane el encuentro, considerando que A necesita i partidos para ganar y B necesita j . Entonces: si $i = 0$ y $j > 0$ entonces A ganó el encuentro, es decir, $p(i, j) = 1$. Si $i > 0$ y $j = 0$ entonces B ganó el encuentro $p(i, 0) = 0$. Si $i, j > 0$ deberá jugarse un partido más y los dos ganan la mitad de las veces. Luego, $p(i, j) = \frac{1}{2}(p(i-1, j) + p(i, j-1))$ si $i, j > 0$. Aquí $p(i-1, j)$ es la probabilidad de A gane el encuentro, si gana el siguiente partido y $p(i, j-1)$ probabilidad de que A gane el encuentro aunque pierda el siguiente partido

1. Escriba un algoritmo recursivo para calcular $p(i, j)$.
2. Determine la complejidad de este algoritmo. Para determinar la complejidad del algoritmo asuma $k = i + j$.

3. Obtenga un algoritmo iterativo (utilizando programación dinámica) para calcular $p(i, j)$. Como una guía para construir su algoritmo complete la tabla (para $n = 4$ partidos, es decir el encuentro lo gana el equipo que primero gana 4 partidos) que se indica a continuación en la que se propone calcular y almacenar las soluciones de los problemas pequeños. Por ejemplo si $i = j = 1$, $p(1, 1) = \frac{1}{2}$.

i/j	0	1	2	3		4
0	1	1	1	1	1	1
1	0	$\frac{1}{2}$	$\frac{3}{4}$			
2	0	$\frac{1}{4}$				
3	0					
4	0					

4. Obtenga la complejidad de este nuevo algoritmo